

Niyet ≈ Kod

Ruby ve Ruby on Rails ile AI çağında doğrulanabilir yazılım

YZM379 · Samsun Üniversitesi · Ekim 2026

Darboğaz artık yazmak değil, doğrulamak

Ajan 10.000 satırı bir oturumda yazar. Soru şu:

Ajanın attığı pası senin koşunda alabilmek için kod nasıl okunmalı?



Bölüm 1 · Ruby
Nesneler, bloklar, Range, DSL, metaprogramming



Bölüm 2 · Rails
CoC, scaffold, model DSL'i, scope, enum, test



Bölüm 3 · Token ekonomisi
Çıktı penceresi, Astra, SaaS'ın maliyeti



Ödev
Fortune 500 SaaS'ının aort dilimini klonla

AI çağı notu Arda Turan'ın kullanılabilir pası: değerli olan ajanın ürettiği değil, senin kullanabildiğin çıktı.

Bölüm 1 · Temel Ruby

Dil, niyeti ne kadar az çeviriyle taşıyor?

Her şey nesne; sayıya bile mesaj gönderirsin

```ruby
5.times { |i| puts i }      # döngü değil, mesaj
3.even?                     # => false
nil.to_a                    # => []
"samsun".capitalize         # => "Samsun"
[3, 1, 2].sort.reverse      # => [3, 2, 1]
10.clamp(1, 5)              # => 5
```

Fonksiyon değil, mesaj. Nesne.metot okuma sırası, cümle sırasına yakın.

? ile biten metot her zaman soru sorar ve true/false döner.

! ile biten metot tehlikelidir: nesneyi değiştirir veya hata fırlatır.

AI çağı notu ? ve ! adlandırma kuralları ajana sözleşme verir; review'da yan etkili satır tek bakışta görünür.

Bloklar ve Enumerable: döngü yazmadan rapor

```ruby
tickets.map(&:status).tally
# => {"open"=>12, "solved"=>30}

tickets.group_by(&:priority)
invoices.sum(&:total)
paid, unpaid = invoices.partition(&:paid?)
emails = users.filter_map { |u| u.email if u.active? }
rows.each_slice(100) { |batch| import(batch) }
```

Prompt cümlesi → metot

“duruma göre say” → **`tally`**

“önceliğe göre grupla” → **`group_by`**

“ödenenleri ayır” → **`partition`**

“aktiflerin e-postası” → **`filter_map`**

“100'erli işle” → **`each_slice`**

AI çağı notu Döngü + sayaç yerine tek metot: yan etki yüzeyi küçülür, ajanın hatası daha az yerde saklanır.

Symbol, Hash ve keyword argüman: adlandırılmış niyet

```
status = :open                          # sabit, ucuz etiket
ticket = { id: 7, status: :open, priority: :high }
ticket[:status]                         # => :open
ticket.fetch(:owner, "atanmamış")       # yoksa varsayılan

def assign(ticket, to:, notify: true)
  # to: zorunlu, notify: isteğe bağlı
end
assign(ticket, to: agent)               # çağrı kendini anlatır
```



Neden önemli?
assign(t, agent, true) mı, assign(t, to: agent) mı? İkincisini ajan da sen de yanlış okuyamazsın. Rails'in tüm DSL'i keyword argüman üzerine kurulu.

AI çağı notu Keyword argüman, prompt'taki adlandırılmış parametrenin koddaki ikizidir.

Range: “arasında” ve “o günden beri” tek ifade

```
(1..10).include?(5)          # => true
(1...10).to_a.last           # => 9   (üç nokta: son hariç)
age = 17
case age
when 0..17  then "reşit değil"
when 18..   then "yetişkin"      # sonsuz Range
end
(Date.new(2026, 10, 1)..Date.new(2026, 10, 31)).count(&:saturday?)
```

AI çağı notu Sonsuz Range (18..) Rails'te where(created_at: 1.week.ago..) olarak geri dönecek.

nil ile barışık kod: &. , ||= ve then

`&.` Güvenli gezinme

```
user&.profile&.city
# user nil ise nil döner,
# patlamaz
```

`||=` Yoksa ata

```
@cache ||= load_rates
# ilk çağrıda yükler,
# sonra hatırlar
```

`then` Zincire sok

```
params[:q]
  .then { |q| q.strip }
  .then { |q| search(q) }
```

Uyarı: &. hatayı saklayabilir. “Kullanıcı yoksa ne olmalı?” sorusunun cevabı domain kararıdır; operatör bu kararı vermez.

AI çağı notu Ajan &. serpiştirmeyi sever: her &. için “nil burada meşru mu?” diye sor.

Pattern matching: webhook'u bildirerek ayrıştır

```
case event
in { type: "payment.succeeded", amount:, currency: "TRY" }
  mark_paid(amount)
in { type: "payment.failed", attempt: 3.. }
  cancel_subscription
in { type: "payment.failed" }
  schedule_retry(in: 3.days)
end
```

Şekli tarif edersin, if/else merdiveni kurmazsın.

`amount:` eşleşirken değişkene bağlar.

`attempt: 3..` Range ile koşul.

Eşleşme yoksa NoMatchingPatternError: sessizce geçmez.

AI çağı notu Stripe benzeri ödev dilimlerinde webhook işleme bu yapıyla okunur kalır.

Sınıf, modül ve tek satırlık metot

```ruby
class Invoice
  attr_reader :lines, :due_on

  def initialize(lines:, due_on:)
    @lines, @due_on = lines, due_on
  end

  # endless method (Ruby 3.0+)
  def total    = lines.sum(&:amount)
  def overdue? = due_on < Date.today
end

module Payable
  def pay! = update!(status: :paid)
end
```



Sınıf
Durum + davranış. attr_reader okuma kapısı açar.



Modül
Davranışı karıştır (include). Rails'te concern olur.



def x = …
Ruby 3.0+: tek ifadeli metot, iş kuralı tek satır.

ActiveSupport: 1.week.ago aslında burada

```ruby
require "active_support/all"    # düz Ruby'de de çalışır

1.week.ago
3.days.from_now
Time.current.beginning_of_month
due_at.past?
trial_ends_at = created_at + 14.days
"invoice".pluralize             # => "invoices"
[].blank?                       # => true
%w[a b c].to_sentence           # => "a, b, and c"
```



Canlı tuzak

Ajana “3 iş günü sonra” de. Çoğu zaman şunu yazar:

```ruby
3.business_days.from_now
```

Konsolda çalıştır: NoMethodError. Rails'te yok; business_time gem'i gerekir. Uydurulan API'nin tipik örneği.

AI çağı notu Okunur API ajanın uydurmasını da okunur kılar: yanlış satır, doğal dilde yanlış cümle gibi göze batar.

“DSL doğal dille 1:1 eşleşir” — doğru mu?

Niyet: *“Bir hafta önceki zamanı ver.”*

Python

```
from datetime import datetime, timedelta
datetime.now() - timedelta(weeks=1)
```

Ruby + ActiveSupport

```
1.week.ago
```

Doğru olan

Sözcük sırası niyetin sırası. Okuyan insan, ajanın satırını cümle gibi denetler; yanlış satır yanlış cümle gibi göze batar.

Abartılı olan

1:1 eşleşme İngilizceyle. Türkçe prompt'ta bir çeviri katmanı var. “Neredeyse 1:1” bir ölçüm değil, slogan; token farkını kendin ölç (G1).

AI çağı notu DSL'in değeri yazma hızı değil, doğrulama hızı: niyeti okuyabildiğin satırı onaylayabilirsin.

DSL sihir değil: parantezsiz metot + blok

```ruby
# config/routes.rb  — bir DSL gibi görünür
resources :tickets do
  member { post :resolve }
end

# aslında sadece Ruby metot çağrıları:
resources(:tickets) do
  member() { post(:resolve) }
end
```

```ruby
# kendi mini DSL'imiz
class Sla
  def self.rule(priority, within:)
    (@rules ||= {})[priority] = within
  end
  rule :high,   within: 4.hours
  rule :normal, within: 1.day
end
```

AI çağı notu has_many, validates, scope, resources: hepsi sınıf metodu. Ajanın ürettiği DSL de sadece Ruby; okunabilir ve test edilebilir.

Metaprogramming: kodu yazan kod

```ruby
class Ticket
  STATUSES = %i[open pending solved]

  STATUSES.each do |st|
    # 3 metot doğar: open?, pending?, solved?
    define_method("#{st}?") { status == st }
  end
end

Ticket.new.respond_to?(:solved?)    # => true
# ama dosyada "def solved?" diye bir satır YOK
```

Rails'te her gün

`has_many :line_items`

line_items, line_items<<, line_item_ids… üretir

`enum :status`

paid?, paid!, Invoice.paid üretir

Konsolda gör:

`Invoice.instance_methods.grep(/paid/)`

AI çağı notu Boilerplate'i yok eden güç bu. Bedeli: üretilen metot dosyada yazmaz; ajan da sen de grep'le bulamazsın.

Sihir vergisi: görünmeyen kodu ajan da göremez

```ruby
class Settings
  def method_missing(name, *args)
    return store(name, args[0]) if name.end_with?("=")
    fetch(name)
  end
end
# bu metot hiçbir yerde tanımlı değil:
Settings.new.smtp_host

class Integer    # monkey patch
  def percent_of(x) = x * self / 100.0
end
```

grep'te yok
Ajan dosyada aradığı metodu bulamaz; ya uydurur ya yanlış yere ekler.

Davranış sapması
Monkey patch, ajanın eğitim verisindeki kütüphane davranışını değiştirir.

Kural
Ödevde method_missing ve global monkey patch yok. Gerekirse README'de listele.

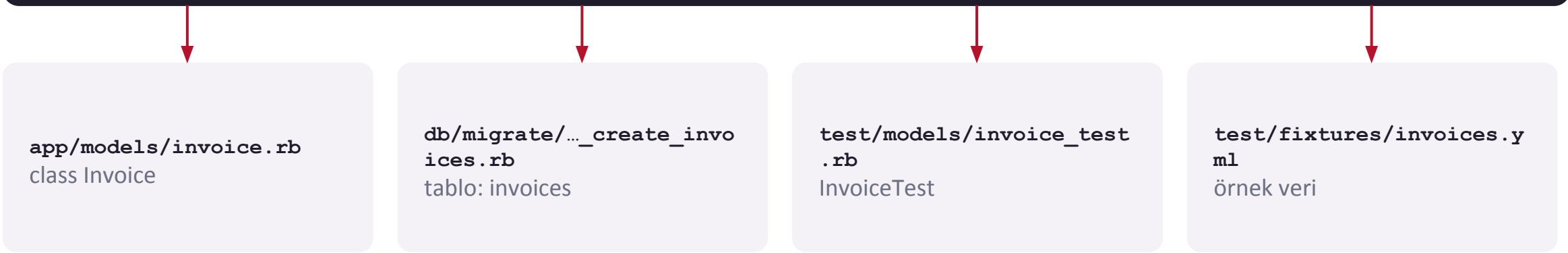
AI çağı notu Görünen DSL ajanın dostu, görünmeyen sihir ajanın kör noktası.

Bölüm 2 · Ruby on Rails

Konvansiyon, ajanın tahmin alanını daraltır

Convention over Configuration: ajan sormaz

```
bin/rails g model Invoice account:references total:decimal due_on:date
```



Ajan sormaz: InvoicesController, invoices tablosu, invoice_path… adlar dünya genelinde aynı.

Bağlam tasarrufu: mimariyi anlatmak için binlerce satırı ajana okutmazsın.

AI çağı notu Konvansiyondan sapmak serbest; ama her sapma AGENT.md'ye yazılmak zorunda.

Scaffold: 30 saniyede çalışan CRUD

```
bin/rails g scaffold Ticket subject:string status:integer sla_due_at:datetime
bin/rails db:migrate && bin/rails server
```

Model + migration
Ticket, tickets tablosu

Controller
7 eylem: index … destroy

View'lar
liste, form, detay

Route
resources :tickets

Testler
controller testleri



Scaffold = iskelet, aort dilimi değil. Status bir integer, geçiş kuralı yok, SLA hesabı yok. Ödevde scaffold ile başlamak serbest; scaffold ile bitirmek 0 puan. Fark, scaffold'un üstüne eklediğin domain kuralları.

AI çağı notu CRUD'u ajan bedava üretir; değer, ajanın tahmin edemeyeceği iş kuralındadır.

Model DSL'i: iş kuralları bildirim olarak

```ruby
class Invoice < ApplicationRecord
  belongs_to :account
  has_many   :line_items, dependent: :destroy

  validates :due_on, presence: true
  validates :number, presence: true,
                     uniqueness: { scope: :account_id }
  validates :total,
            numericality: { greater_than_or_equal_to: 0 }
end
```

Türkçe kural → Rails satırı

“Faturanın bir hesabı var”

`belongs_to :account`

“Kalemler faturayla silinir”

`dependent: :destroy`

“Numara hesap içinde tekil”

`uniqueness: { scope: … }`

“Tutar eksi olamaz”

`greater_than_or_equal_to: 0`

AI çağı notu Domain modeli tek ekranda okunur: ajanın unuttuğu kural, eksik satır olarak görünür.

Scope zinciri: domain kelimeleri sorguya girer

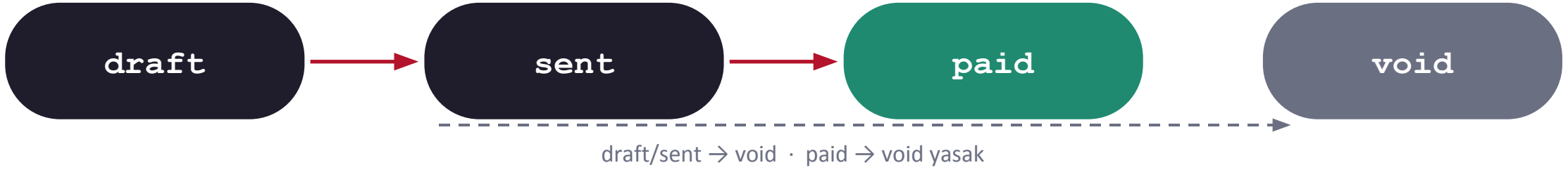
```
class Ticket < ApplicationRecord
  scope :open_ones, -> { where(status: %i[new open pending]) }
  scope :overdue,   -> { where(sla_due_at: ...Time.current) }
  scope :recent,    -> { where(created_at: 1.week.ago..) }
  scope :for,       ->(agent) { where(assignee: agent) }
end

Ticket.open_ones.overdue.for(current_agent).recent
```

Prompt: *“Bu ajanın son bir haftadaki, SLA'sı geçmiş açık talepleri.”* **Son satır bu cümlenin kendisi.**

AI çağı notu SQL'i ajan yazar, ama niyet modelde adlandırılmış olarak kalır; review scope adlarını okur.

enum ile durum makinesi: geçişler görünür olsun




```
class Invoice < ApplicationRecord
  enum :status, { draft: 0, sent: 1, paid: 2, void: 3 }

  def pay!
    raise InvalidTransition unless sent?
    paid!
  end
end
```

`enum` sent?, paid!, Invoice.paid üretir.

Ama **`paid!`** her durumdan çalışır; geçiş kuralını sen yazarsın.

Yasak geçiş testi yoksa ajanın atladığı kural yakalanmaz.

Migration: şema değişikliği, çalıştırılabilir tarihçe

…_create_tickets.rb
```
create_table :tickets
```

…_add_priority.rb
```
add_column :tickets, :priority, :integer
```

…_add_sla_due_at.rb
```
add_column :tickets, :sla_due_at, :datetime
```

…_index_sla.rb
```
add_index :tickets, :sla_due_at
```

Okunabilir tarih: domain'in nasıl büyüdüğünü dosya adları anlatır.

Geri alınabilir: bin/rails db:rollback

Ajan kuralı: çalışmış migration düzenlenmez, yenisi yazılır.

AI çağı notu SLA eklemek istediğinde ajana önce migration'ı yazdır, sonra modeli: sıra niyeti korur.

Rails 8: tek kişi + ajan canlıya çıkar

Kimlik doğrulama
bin/rails g authentication

Solid Queue
Gece işleri, retry, SLA kontrolü; Redis yok

Solid Cache / Cable
Önbellek ve canlı yayın veritabanında

Hotwire (Turbo)
Canlı güncellenen liste, JS build yok

Kamal
Kendi sunucuna tek komutla deploy

Konsol + generator
bin/rails console ile her kuralı dene

AI çağı notu Daha az parça, ajanın bozabileceği daha az yüzey demek. Ödevde Render benzeri yeni bir platformu deneyeceksiniz.

Dinamik dilde test, tip sisteminin yerini tutar

```
test "SLA geçince talep gecikmiş sayılır" do
  ticket = tickets(:new_high_priority)    # fixture
  travel_to 5.hours.from_now do
    assert ticket.reload.overdue?
  end
end

test "ödenmiş fatura iptal edilemez" do
  invoice = invoices(:paid)
  assert_raises(InvalidTransition) { invoice.void! }
end
```



Ödev kuralı

CI rozeti yoksa “testler geçti” cümlesi geçersizdir.

Ajanın yazdığı test raporu kanıt değildir; çalışan CI kanıttır.

AI çağı notu travel_to ile zamanı oynatabilmek, 1.week.ago ailesini test edilebilir kılar.

Sihir vergisi Rails'te: callback ve default_scope

Yapı	Ajana ne yapar	Ödev kuralı
`before_save / after_commit`	Yan etki modelin içinde saklanır; ajan düzeltmeyi yanlış yere yapar	Domain işlemi açık metot/servis; callback sadece türetilmiş alan için
`default_scope`	Ajanın yazdığı her sorgu sessizce filtrelenir	Yasak. Adlandırılmış scope kullan
`concern'de gizli davranış`	Metot modelde görünmez, başka dosyadan gelir	Concern adı davranışı anlatmalı; AGENT.md'de listelenir
`Dinamik tip`	Hata çalışma anında çıkar	Test = tip sistemi; CI kırmızıysa özellik yok sayılır

AI çağı notu Konvansiyon tahmini daraltır, gizli sihir tahmin gerektirir. İkisi aynı framework'te yaşar.

AGENT.md: ajanla ortak sözlük

Domain sözlüğü TR ↔ EN
talep = Ticket, SLA ihlali = overdue, aort dilimi = core flow. 1.week.ago İngilizce niyete eşleşir, Türkçeye değil.

Yasak sihirler
default_scope, method_missing, iş mantığı içeren callback.

Test kuralı
Her zaman kuralı travel_to testli, her yasak geçiş testli.

Konvansiyondan sapmalar
Nerede, neden saptın? Yazılmayan sapma ajanı yanıltır.

AI çağı notu Şablon verilmez: AGENT.md'yi ajanla birlikte üretmek dersin kendisi. İlk ve son sürümün diff'i teslim edilir.

Bölüm 3 · Token ekonomisi

Okuma penceresi büyüdü; asıl değişim yazma penceresinde

Bağlam okur, çıktı yazar: darboğaz çıktı penceresi

Bağlam penceresi

~1M token

Ajanın tek seferde OKUYABİLDİĞİ

Bugünkü çıktı

~128k token

≈ 12.800 satır kod

Çıktı 1M olursa

~1M token

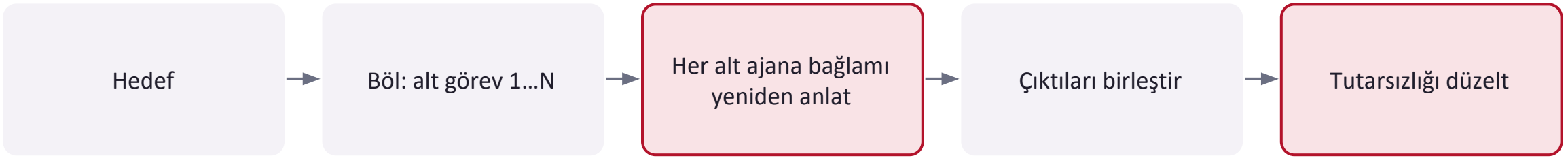
≈ 100.000 satır, tek oturum

Varsayım: satır başına ~10 token. Derste “2.000 sayfa / 200.000 satır” dedik; o hesap sayfa başına 100 satır varsayıyordu. Varsayımı değiştir, sonuç 2 kat oynar: rakamı değil büyüklük sırasını aklında tut.

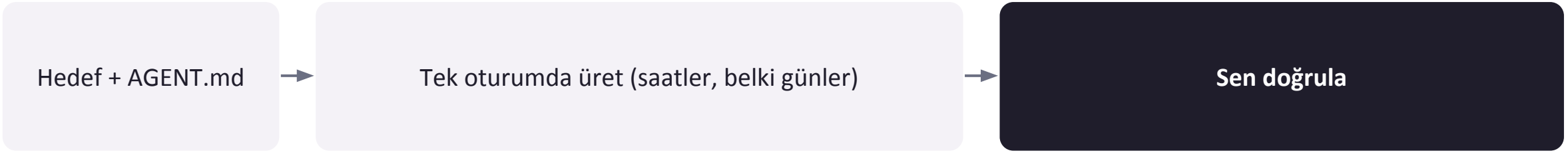
AI çağı notu Pencereler büyüdükçe kod üretimi ucuzlar; okuyup onaylama ucuzlamaz. Ruby'nin kısalığı tam bu ikinci maliyete dokunur.

Görev bölmenin gizli maliyeti: anlatma vergisi

Bugün: küçük çıktı penceresi



Büyük çıktı penceresi



Kırmızı kutular token ve zaman yakar: bağlamı her alt ajana yeniden anlatmak, sonra parçaları uyuşturmak.

AI çağı notu Darboğaz son kutuya taşınıyor. 100.000 satırı kim, kaç saatte doğrulayacak?

Astra'nın iddiası: “Bunu yapamadım” diyebilen ajan



Kavramsal eğri (ölçüm değil)

Bugünün tipik hatası: uygulamayı ayağa kaldıramayan ajan, 100 testten 70'i geçti diyen bir rapor uydurur.

Astra'nın iddiası: doğrulayamadığı yerde uydurmak yerine durur ve söyler. Doğrulanmış bir sonuç değil, bir iddia.

Bize dersi: güveni doğru kalibre edilmiş ajan da, öğrenci de “bilmiyorum” noktasını işaretleyebilir.

AI çağı notu Ödevde de aynı kural: Yapamadıklarım listesi puan kaybettirmez; uydurulan rapor notu sıfırlar.

Token ekonomisi: Ruby avantajı bir hipotez, ölçün

Bir özelliğin gerçek maliyeti

```
maliyet = çıktı token × fiyat  +  düzeltme turları  +  doğrulama süresi × saatin
```

Ölçüt	Rails (sen)	Seçtiğin diğer yığın (sınıf arkadaşın)
Çıktı token (ajan arayüzünden)		
Düzeltme turu sayısı		
CI'da yeşil test sayısı		
Yakalanan uydurma (G5)		
Kodu okuyup onaylama süresi (dk)		

AI çağı notu Benchmark'a tapmayın: tek görev, tek ajan, eğitim verisi etkisi. Kendi ölçümünüzü yanlılıklarıyla birlikte raporlayın.

SaaS'ın kalesi: kopyalamanın maliyeti çöktü mü?

~10:1

çalışan : SaaS

Derste konuştuğumuz kaba oran: 1.000 çalışanlı firma ≈ 100 SaaS ürünü

300–500k

satır

Orta ölçekli bir SaaS'ın kaba kod büyüklüğü (ders iddiası)

günler

+ token faturası

Coding agent ile klon süresi ve maliyeti, yıllar ve lisans yerine (ders iddiası)



Ama: kopyalamak ucuzladı; yeni ihtiyacı görüp topu oraya atmak ucuzlamadı. Firma bugünü klonlayabilir, yarının ihtiyacını tasarlayan yazılımcı hâlâ kıt. Ödevdeki 100+ tam burası.

AI çağı notu SaaSpocalypse tezi: değer koddan, domain'i anlayıp kullanılabilir pas atma becerisine kayıyor.

Ödev · Bir haftada aort dilimi

Fortune 500 şirketlerinin para ödediği bir SaaS'ın ana iş akışını Rails ile klonla

Aort dilimi: ürüne para ödeten tek akış

Tüm ürünü değil, müşterinin o ürünü bırakamamasının nedeni olan akışı uçtan uca kur. En az bir zaman kuralı ve bir durum makinesi içermeli.

İlham	Aort dilimi	Zaman kuralı
Zendesk	Talep → ata → yanıtla → çöz	İlk yanıt SLA'sı
Salesforce	Lead → fırsat → kazanıldı	30 gün dokunulmayan fırsat
Workday (izin)	Talep → onay → bakiye	Yıllık hak ediş
DocuSign	Zarf → imza sırası → tamam	7 günde süre dolması
Stripe Billing	Plan → abonelik → fatura	Deneme süresi, 3 günlük retry
Greenhouse	İlan → aday → teklif	Aşamada bekleme süresi

AI çağı notu Kendi ürün adın ve logon. Orijinal marka sadece README'de ilham kaynağı olarak geçer.

Rubrik: 0 / 85 / 100+ — ara puan yok

0

- Canlı URL yok
- Aort dilimi uçtan uca çalışmıyor
- “Var” denip çalışmayan özellik
- CI'sız “testler geçti”
- Orijinal marka/logo uygulamada

85

- Aort dilimi canlıda uçtan uca
- CI yeşil
- 3 zaman kuralı travel_to testli
- Durum makinesi + yasak geçiş testi
- G1–G5 + Yapamadıklarım bölümü

100+

- 85'in tamamı, artı:
- Kullanılabilir pas: orijinalde olmayan bir adaptasyon
- Kim, neden, nasıl kullanır tek cümlede
- Örn. e-Fatura durumu, akademik takvime bağlı SLA

AI çağı notu “Beceremedim” yazmak not düşürmez; gizlenen bir eksik notu sıfırlar.

5 dakikalık sunum

0:30

Hangi SaaS, hangi aort dilimi, orijinali kim neden öder

2:00

Canlı URL'de aort dilimi baştan sona

1:00

Bir zaman kuralının ve bir durum geçişinin kodu (G1 tablosundan)

1:00

Adaptasyon (100+ iddiası) veya yapamadıklarım

0:30

Kod yeri sorusu: eğitmen bir kural değişikliği söyler, sen dosya/satırı gösterirsin

AI çağı notu Örnek kod yeri sorusu: “SLA hafta sonunu saymasın.” Uygulamazsın; değişikliğin nereye dokunacağını bilirsin.



Ajan hızlı yazar. Sen hızlı doğrula.

Teslim: gelecek ders · canlı URL + GitHub + docs/ (G1–G5) + AGENT.md

Aort dilimini bu ders bitmeden yazılı bildir.